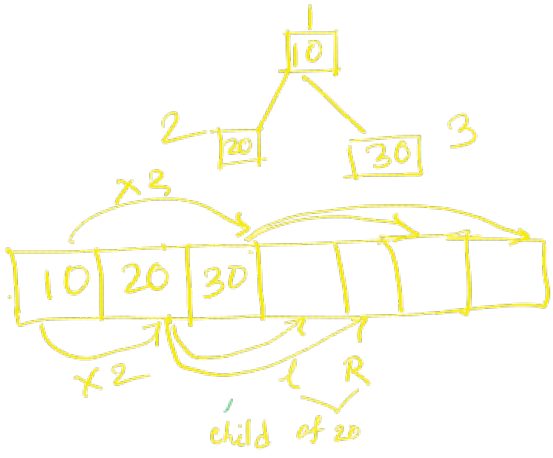
max heap → root of tree = max val
min " " " min "
↳ continuously need max val
→ O(1)
instead of linear search O(N)

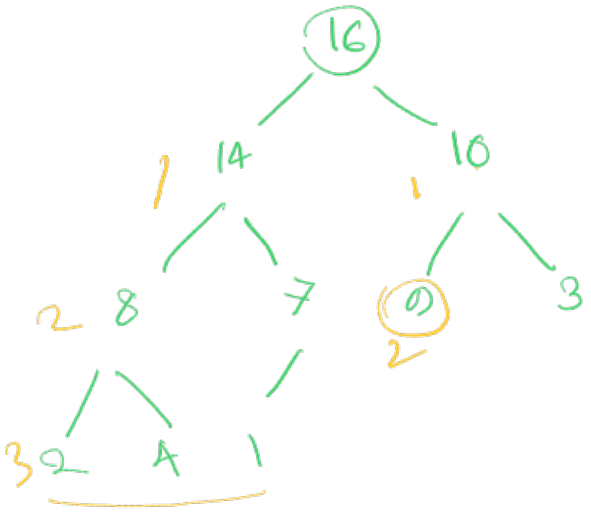
Find 2nd highest (O(log n))

any binary tree can be represented using an array.



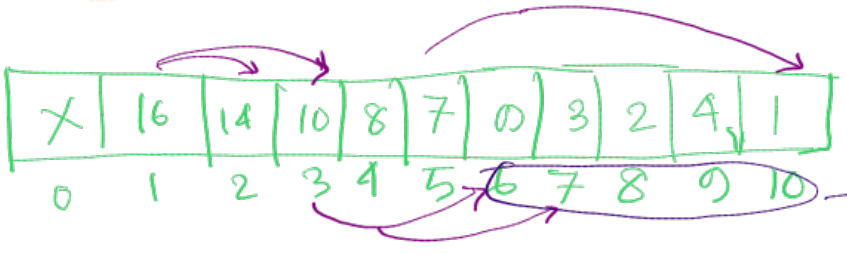
finding any location → O(log n)

max heap = All parents ≥ children



→ Max heap

→ height : 3



→ beyond capacity of array so no children

If using 1 based indexing,

For any node (i) → parent $\lfloor \frac{i}{2} \rfloor$ location

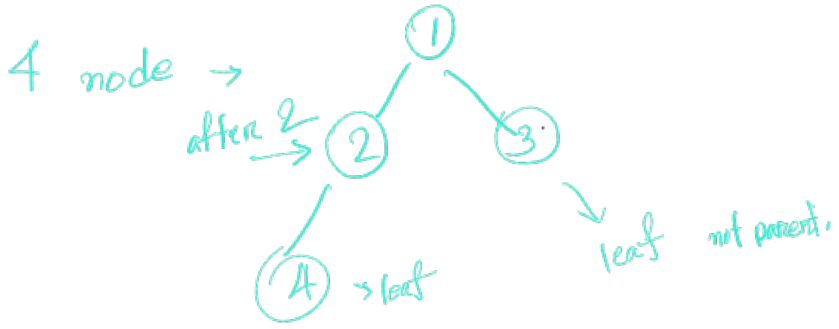
↑

↳ left child $2i$ th index

right $(2i+1)$

After

* $\left(\frac{i}{2}\right)^{th}$ location → everyone is leaf node



Height of tree: from root to farthest leaf node, how many edges.

Height of leaf node: 0.

→ $O(\lfloor \log n \rfloor)$

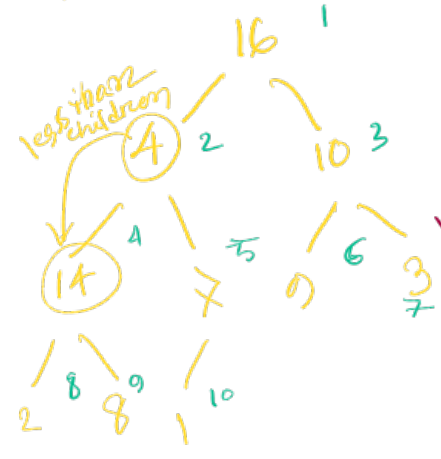
3 →  $h \rightarrow \lfloor \log_2 3 \rfloor = 1.$

7 → 2

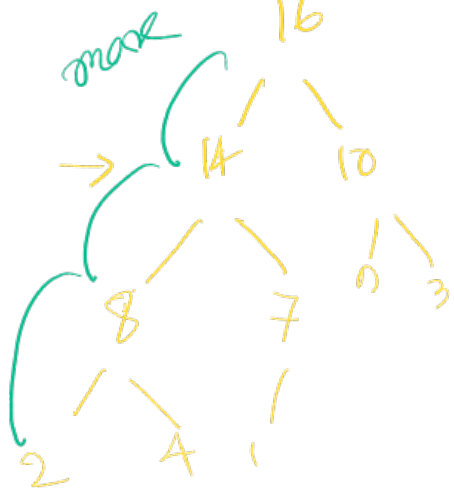
15 → 3

$(\log_2 16) = 4$

heapify location.







Pseudo Code:

max_heapify (A, i)

max heapify (A, i)
↓ Array ↓ tree

$l = 2i$

$r = 2i + 1$

if $A[l] > A[i]$ and $l \leq$ heapsize
largest $= l$ else largest $= i$
if $A[r] > A[\text{largest}]$ and $r \leq$ heapsize
largest $= r$

if largest $\neq i$ (violation)
exchange $A[i], A[\text{largest}]$
max-heapify (A, largest) → complexity → $O(n \log n)$

$O(\log n)$

$O(n \log n)$

↓

traversing
n times

max heapify (A, 2)
swap A[2], A[4] → 4 → 14

max-heapify (A, 4)
largest = 9
swap A[4], A[9] → 4 → 8

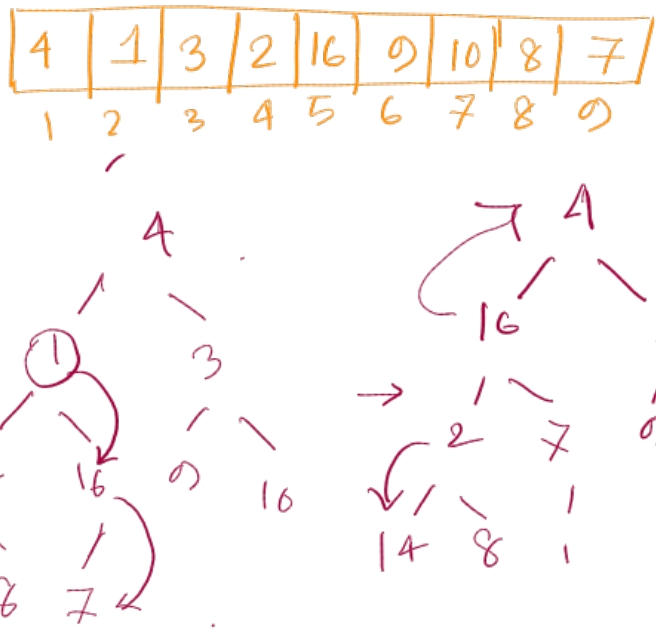
max heapify [A, 9]
largest = i = 9

$O(n(\log n))$

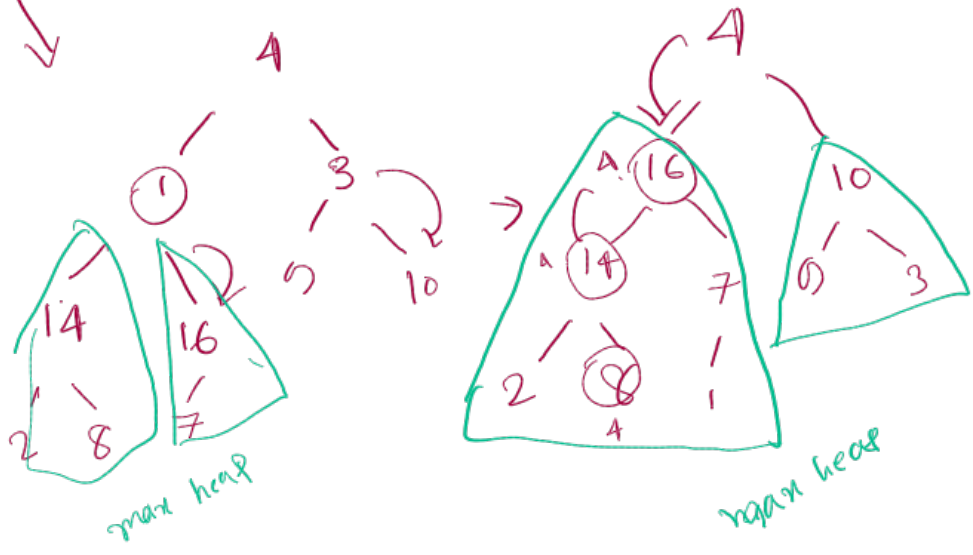
↓

For heaping the whole tree.

An arbitrary set of array is given. U wanna make heap of that.



bottom to top: approach; ✓ not top down



Pseudo

build max_heap(A)

A.heapsize.

for i = length/2 down to 2

max heapify (A, i)

for every heapify → logn and n traversal → nlogn

complexity: nlogn

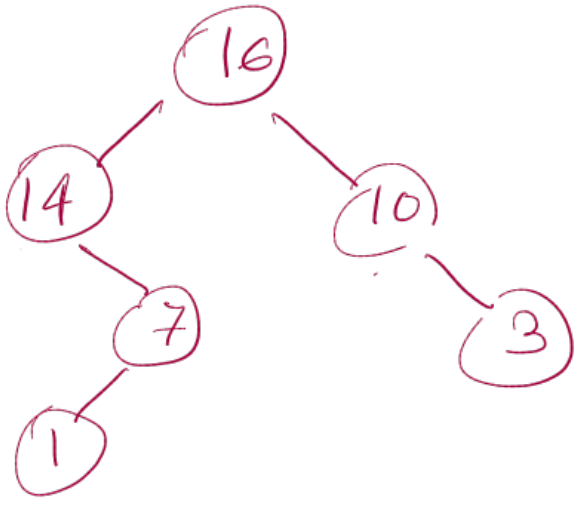
↳ actually O(N) – linear.

17.9.24

Complete binary tree: filled on all levels except lowest ones.

Each parent <= children → min-heap

" >= " → max "



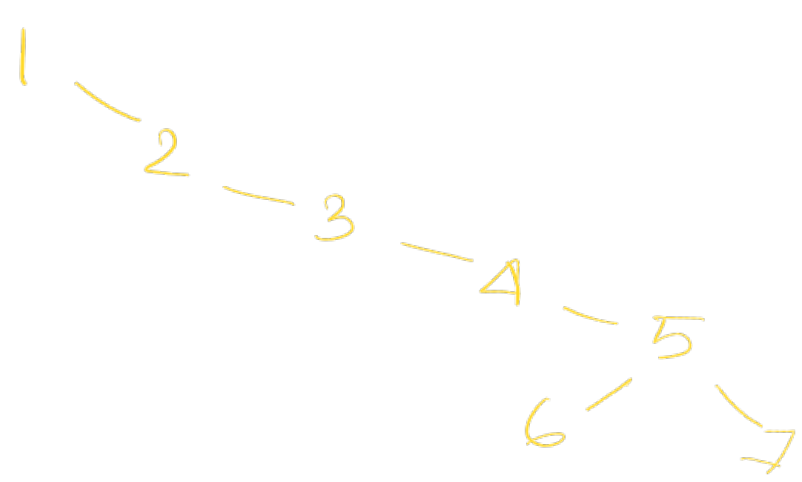


after $\frac{n}{2}$ → everything leaf node

height → for n nodes in complete binary tree,

$$h = O(\log_2 n)$$

Exception:

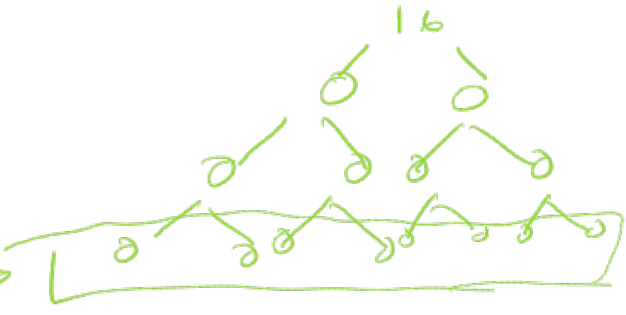


height: N

level = 2^l nodes

1st level → 2

3rd " → $2^3 = 8$



max nodes upto level: $2^{l+1} - 1$

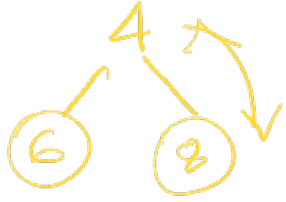
for 3rd = $2^{3+1} - 1$

= 15 total.

heapify (ith node) → Fixing a particular node. to follow heap pattern.

↓

recursive approach.



biggest of the childs will get swapped.

Function:

```
max_heapify (i) {
    l = 2i
    r = 2i+1;
    if  l <= heapsize & heap[l]  > heap[i]
        largest = l
    else
        largest = i
now right child check

if largest != i
    swap heap[i] with heap[largest]
    max-heapify (largest)  -> O(log N)
*   largest = i ;
```

O(1)

O(N).

Why bottom up is correct?

=> left & right childs are max values of the sub-tree.

build_max_heap()

heap. slide + next slide.

25th sept Quiz